

Programación funcional en resumen

Desde un punto de vista sintáctico, un lenguaje funcional puro no tiene comandos —que modifican el estado mediante efectos laterales... sólo expresiones

Dos constructos principales para **definir** expresiones:

- abstracción —permite la construcción de una expresión
- aplicación de una expresión a otra expresión

No hay restricciones sobre:

- pasar funciones como argumentos a otras funciones
- devolver funciones como resultado de otras funciones

⇒ Hay homogeneidad perfecta entre programas y datos

Un programa es una serie de definiciones de valores:

- cada una inserta una nueva asociación en el ámbito
- ... y puede requerir la evaluación de expresiones complejas

Funciones de nivel superior y funciones recursivas

- convierten al mecanismo en flexible y poderoso

Dos operaciones para **evaluar** (semántica de la computación):

- búsqueda de un identificador en el ámbito —cuando se lo encuentra, reemplazar el identificador por su definición
- aplicación de una expresión funcional a un argumento

Cuando unas funciones son devueltas como valores por otras funciones, el ámbito local debe ser preservado por tiempo ilimitado:

- uso extensivo de recolección de basura



“...There's a sign on the wall, but she wants to be sure
'Cause you know sometimes words have two meanings...”

Estudio de los lenguajes de programación:

- estudio de la sintaxis
- estudio de la **semántica**

La forma de las expresiones, sentencias y unidades de un programa

P.ej., la sintaxis de la sentencia **while** en Java es
`while (boolean_expr) statement`

El significado de esas expresiones, sentencias y unidades

P.ej., la semántica de la sentencia **while** es que
Cuando el valor actual de *boolean_expr* es verdadero,
la sentencia *statement*, subordinada, es ejecutada
... y el control vuelve (implícitamente) a *boolean_expr*
para repetir el proceso.

Si *boolean_expr* es falsa, entonces el control es transferido a la sentencia que sigue al **while**

La descripción de la **semántica** de un lenguaje de programación es más compleja que la de la sintaxis:

- problemas técnicos
- la necesidad de mediar entre exactitud y flexibilidad

Se necesita una descripción precisa, no ambigua de lo que debe esperarse de cada constructo sintácticamente correcto

... pero esta descripción no debe anticipar elecciones propias de la implementación y que no son parte del lenguaje

La descripción de la **semántica** de un lenguaje de programación es más compleja que la de la sintaxis:

- problemas técnicos
- la necesidad de mediar entre exactitud y flexibilidad

Se necesita una descripción precisa, no ambigua de lo que debe esperarse de cada constructo sintácticamente correcto

... pero esta descripción no debe anticipar elecciones propias de la implementación y que no son parte del lenguaje

Debe poder saberse antes de la ejecución del programa, e independientemente de la arquitectura, **qué va a pasar a la hora de la ejecución**

Debe **permitir diferentes implementaciones del lenguaje**, todas correctas con respecto a la semántica

No es difícil lograr exactitud:

- la semántica se puede describir usando un compilador específico y una arquitectura específica
- el significado “oficial” de un programa corresponde a su ejecución en esta arquitectura después de ser traducido por este compilador

... pero a costa de flexibilidad:

- existe sólo una implementación

... pero, ¿hasta qué nivel de detalle esta implementación es normativa?

No es difícil lograr exactitud:

- la semántica se puede describir usando un compilador específico y una arquitectura específica
- el significado “oficial” de un programa corresponde a su ejecución en esta arquitectura después de dser traducido por este compilador

... pero a costa de flexibilidad:

- existe sólo una implementación

... pero, ¿hasta qué nivel de detalle esta implementación es normativa?

... y todas las otras que puede haber son totalmente equivalentes

¿Es el tiempo de ejecución de un programa parte de la definición?

Cuando la tecnología de implementación de la arquitectura física cambie, ¿cambia la semántica?

Se propone usar métodos formales para describir semántica

- provenientes de la lógica matemática

... aunque en la práctica la mayoría de las definiciones oficiales de lenguajes usa más bien lenguajes naturales

Los métodos formales son usados en las fases preparatorias del diseño de un lenguaje

... o para describir alguna característica particular del lenguaje en que sea esencial evitar la ambigüedad

Se propone usar métodos formales para describir semántica

- provenientes de la lógica matemática

... aunque en la práctica la mayoría de las definiciones oficiales de lenguajes usa más bien lenguajes naturales

Los métodos formales son usados en las fases preparatorias del diseño de un lenguaje

... o para describir alguna característica particular del lenguaje en que sea esencial evitar la ambigüedad

En **semántica denotacional**, el significado de un programa es dado por una función que expresa el comportamiento input/output del programa, con referencia al ámbito y la memoria

Se propone usar métodos formales para describir semántica

- provenientes de la lógica matemática

... aunque en la práctica la mayoría de las definiciones oficiales de lenguajes usa más bien lenguajes naturales

Los métodos formales son usados en las fases preparatorias del diseño de un lenguaje

... o para describir alguna característica particular del lenguaje en que sea esencial evitar la ambigüedad

En **semántica denotacional**, el significado de un programa es dado por una función que expresa el comportamiento input/output del programa, con referencia al ámbito y la memoria

En **semántica operacional**, el significado de un programa es dado por el comportamiento de un computador abstracto —el intérprete del lenguaje “Análisis de casos exhaustivo”
Hay gran variedad de técnicas operacionales

Se propone usar métodos formales para describir semántica

- provenientes de la lógica matemática

... aunque en la práctica la mayoría de las definiciones oficiales de lenguajes usa más bien lenguajes naturales

Los métodos formales son usados en las fases preparatorias del diseño de un lenguaje

... o para describir alguna característica particular del lenguaje en que sea esencial evitar la ambigüedad

En **semántica denotacional**, el significado de un programa es dado por una función que expresa el comportamiento input/output del programa, con referencia al ámbito y la memoria

En **semántica operacional**, el significado de un programa es dado por el comportamiento de un computador abstracto —el intérprete del lenguaje “Análisis de casos exhaustivo”

Hay gran variedad de técnicas operacionales

En **semántica axiomática**, el significado de un programa es dado por las transformaciones que la ejecución de los comandos produce en el estado, caracterizado por fórmulas de lógica de predicados

Los axiomas son fórmulas a priori verdaderas

Especifica lo que puede ser demostrado acerca del programa

Semántica operacional

- 1) Diseñar un lenguaje intermedio apropiado (un lenguaje de máquina es de muy bajo nivel) —se busca claridad:
 - Cada constructo debe tener un significado obvio no ambiguo
- 2) Construir una máquina virtual (intérprete) para el lenguaje intermedio, para ejecutar comandos individuales, segmentos de código, o programas completos

Semántica operacional

- 1) Diseñar un lenguaje intermedio apropiado (un lenguaje de máquina es de muy bajo nivel) —se busca claridad:
 - Cada constructo debe tener un significado obvio no ambiguo
- 2) Construir una máquina virtual (intérprete) para el lenguaje intermedio, para ejecutar comandos individuales, segmentos de código, o programas completos

C Statement

```
for (expr1; expr2; expr3) {  
    ...  
}
```

Ej. usado en textos de programación y manuales de referencia de lenguajes

Meaning

```
    expr1;  
loop: if expr2 == 0 goto out  
    ...  
    expr3;  
    goto loop  
out: ...
```

Semántica operacional

- 1) Diseñar un lenguaje intermedio apropiado (un lenguaje de máquina es de muy bajo nivel) —se busca claridad:
 - Cada constructo debe tener un significado obvio no ambiguo
- 2) Construir una máquina virtual (intérprete) para el lenguaje intermedio, para ejecutar comandos individuales, segmentos de código, o programas completos

`ident = var`
`ident = ident + 1`
`ident = ident - 1`
goto label
if var *relop* var **goto** label
`ident = var bin_op var`
`ident = un_op var`

... en que *relop* es uno de =, <>, >, <, >=, <=
... *ident* es un identificador
... *var* es un identificador o una constante
... *bin_op* es un operador aritmético binario
... *un_op* es un operador unario

```

ident = var
ident = ident + 1
ident = ident - 1
goto label
if var relop var goto label

```

```

ident = var bin_op var
ident = un_op var

```

```

for (count1 = 0, count2 = 1.0;
      count1 <= 10 && count2 <= 100.0;
      sum = ++count1 + count2, count2 *= 2.5);

```

... en que *relop* es uno de =, <>, >, <, >=, <=

... *ident* es un identificador

... *var* es un identificador o una constante

... *bin_op* es un operador aritmético binario

... *un_op* es un operador unario

```

count1 = 0
count2 = 1.0
loop:
  if count1 > 10 goto out
  if count2 > 100.0 goto out
  count1 = count1 + 1
  sum = count1 + count2
  count2 = count2 * 2.5
  goto loop
out: ...

```



```

ident = var
ident = ident + 1
ident = ident - 1
goto label
if var relop var goto label
ident = var bin_op var
ident = un_op var

```

... en que *relop* es uno de =, <>, >, <, >=, <=

... *ident* es un identificador

... *var* es un identificador o una constante

... *bin_op* es un operador aritmético binario

... *un_op* es un operador unario

```

sum = 0;
indat = Int32.Parse(Console.ReadLine());
while (indat >= 0) {
    sum += indat;
    indat = Int32.Parse(Console.ReadLine());
}

value = Int32.Parse(Console.ReadLine());
do {
    value /= 10;
    digits ++;
} while (value > 0);

```

```

loop:
    if control_expression is false goto out
    [loop body]
    goto loop
out: ...

```

```

loop:
    [loop body]
    if control_expression is true goto loop

```

Semántica axiomática

Basada en una *lógica de programación*:

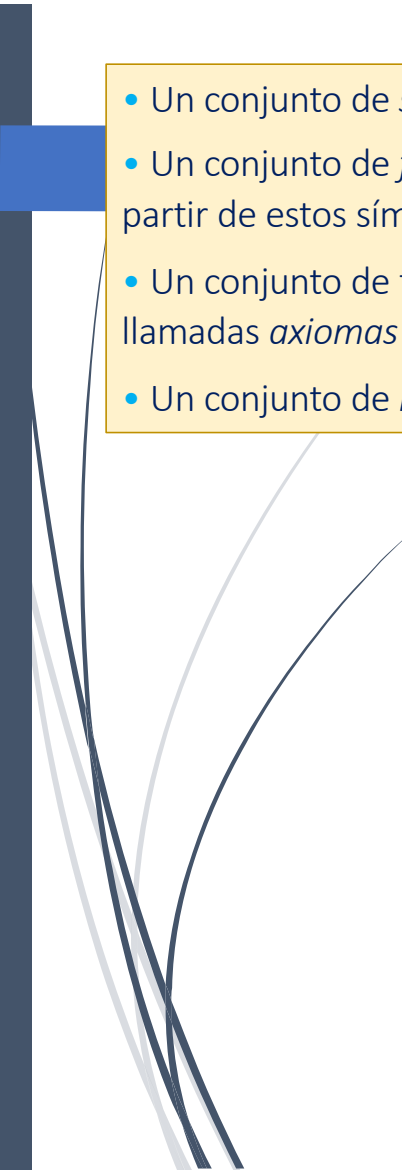
- **Un sistema lógico formal** que facilita hacer afirmaciones precisas acerca de la ejecución de un programa

Semántica axiomática

Basada en una *lógica de programación*:

- Un **sistema lógico formal** que facilita hacer afirmaciones precisas acerca de la ejecución de un programa

- Un conjunto de *símbolos*
- Un conjunto de *fórmulas* construidas a partir de estos símbolos
- Un conjunto de fórmulas distinguidas llamadas *axiomas*
- Un conjunto de *reglas de inferencia*

- 
- Un conjunto de *símbolos*
 - Un conjunto de *fórmulas* construidas a partir de estos símbolos
 - Un conjunto de fórmulas distinguidas llamadas *axiomas*
 - Un conjunto de *reglas de inferencia*

Secuencias de símbolos bien formadas

Fórmulas especiales que se supone a priori verdaderas

Especifican cómo derivar fórmulas verdaderas adicionales a partir de los axiomas y de otras fórmulas verdaderas

- Un conjunto de *símbolos*
- Un conjunto de *fórmulas* construidas a partir de estos símbolos
- Un conjunto de fórmulas distinguidas llamadas *axiomas*
- Un conjunto de *reglas de inferencia*

Secuencias de símbolos bien formadas

Fórmulas especiales que se supone a priori verdaderas

Especifican cómo derivar fórmulas verdaderas adicionales a partir de los axiomas y de otras fórmulas verdaderas

Fórmulas

Cada H_i es un **hipótesis**

C es una **conclusión**

$$\frac{H_1, H_2, \dots, H_n}{C}$$

Si las hipótesis son verdaderas, entonces podemos inferir que la conclusión es verdadera

- Un conjunto de *símbolos*
- Un conjunto de *fórmulas* construidas a partir de estos símbolos
- Un conjunto de fórmulas distinguidas llamadas *axiomas*
- Un conjunto de *reglas de inferencia*

Secuencias de símbolos bien formadas

Fórmulas especiales que se supone a priori verdaderas

Especifican cómo derivar fórmulas verdaderas adicionales a partir de los axiomas y de otras fórmulas verdaderas

Fórmulas

Cada H_i es un **hipótesis**

C es una **conclusión**

$$\frac{H_1, H_2, \dots, H_n}{C}$$

Si las hipótesis son verdaderas, entonces podemos inferir que la conclusión es verdadera

Una **demostración** es una secuencia de líneas, c/u de las cuales es un axioma o puede ser derivada de las líneas anteriores mediante la aplicación de una regla de inferencia

Una lógica de programación

Un sistema lógico formal que permite establecer y demostrar propiedades de programas

Predicados, llaves y comandos del lenguaje

- Un conjunto de *símbolos*
- Un conjunto de *fórmulas* construidas a partir de estos símbolos
- Un conjunto de fórmulas distinguidas llamadas *axiomas*
- Un conjunto de *reglas de inferencia*

Comando o lista de comandos

Triples: $\{P\} S \{Q\}$

Su interpretación caracteriza la relación entre P y Q , y S

Predicados que especifican las relaciones entre los valores de las variables del programa

Comando o lista de comandos

Triples: $\{P\} S \{Q\}$

Predicados que especifican las relaciones entre los valores de las variables del programa

Su *interpretación* caracteriza la relación entre P y Q , y S

El triple $\{P\} Q. \{S\}$ es verdadero si, siempre que la ejecución de S empiece en un estado que satisface P y la ejecución de S termina, entonces el estado resultante satisface Q

Corrección parcial: "... y la ejecución de S termina ..."